# Tutorial 8_B:

## Object Detection using Faster RCNN (Pre-Trained Model)

## Objectives:

- Understand Pre trained model
- Loading pre trained model from tensor flow hub
- Evaluating on a random image

Here's a step-by-step breakdown:

### Step 1 —Import Libraries

We will import the important libraries to get started.

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import cv2
import matplotlib.pyplot as plt
import os
from PIL import Image
```

### Step 2 Load Pretrained Faster R-CNN model

Load the pretrained model and also the class labels that are used for training the model

```
# Step 1: Load the Pre-trained Faster R-CNN Model from TensorFlow Hub
MODEL_URL = "https://tfhub.dev/tensorflow/faster_rcnn/resnet50_v1_640x640/1"
model = hub.load(MODEL_URL)

# Class labels for COCO Dataset (used by the pre-trained model)
COCO_CLASSES = [
    "person", "bicycle", "car", "motorcycle", "airplane", "bus", "train", "truck", "boat", "traffic light",
    "fire hydrant", "stop sign", "parking meter", "bench", "bird", "cat", "dog", "horse", "sheep", "cow", "elephant",
    "bear", "zebra", "giraffe", "backpack", "umbrella", "handbag", "tie", "suitcase", "frisbee", "skis", "snowboard",
    "sports ball", "kite", "baseball bat", "baseball glove", "skateboard", "surfboard", "tennis racket", "bottle",
    "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana", "apple", "sandwich", "orange", "broccoli",
    "carrot", "hot dog", "pizza", "donut", "cake", "chair", "couch", "potted plant", "bed", "dining table", "toilet",
    "TV", "laptop", "mouse", "remote", "keyboard", "cell phone", "microwave", "oven", "toaster", "sink", "refrigerator",
    "book", "clock", "vase", "scissors", "teddy bear", "hair drier", "toothbrush"
]
```

# Step 3: Function for performing object detection

Making a function to perform desired objective

```python
# Function to perform object detection
def detect_objects(image_path, model, threshold=0.5):
    # Debug message to verify the file path
    print(f"Loading image from: {image_path}")

    # Load the image
    image = cv2.imread(image_path)
    if image is None:
        raise FileNotFoundError(f"Could not load image at {image_path}. Please check the file path.")

    # Convert to RGB format for visualization and TensorFlow processing
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Resize image to the model's expected input size
    input_tensor = tf.convert_to_tensor(image_rgb, dtype=tf.uint8)
    input_tensor = input_tensor[tf.newaxis, ...]  # Add batch dimension

    # Run object detection
    detections = model(input_tensor)

    # Extract detection results
    boxes = detections['detection_boxes'][0].numpy()  # Bounding boxes
    class_indices = detections['detection_classes'][0].numpy().astype(int) - 1  # Class indices, -1 to adjust for zero-indexed
    scores = detections['detection_scores'][0].numpy()  # Confidence scores
```

```python
    # Filter out detections below the confidence threshold
    valid_detections = scores >= threshold
    boxes = boxes[valid_detections]
    class_indices = class_indices[valid_detections]
    scores = scores[valid_detections]

    return image_rgb, boxes, class_indices, scores
```

# Step 4: Function for displaying image with bounding box

Develop a function for displaying image with bounding box.

```python
# Function to display the image with detected bounding boxes
def display_image_with_detections(image, boxes, class_indices, scores, threshold=0.5):
    height, width, _ = image.shape
    for i, box in enumerate(boxes):
        if scores[i] >= threshold:
            ymin, xmin, ymax, xmax = box
            (left, right, top, bottom) = (int(xmin * width), int(xmax * width), int(ymin * height), int(ymax * height))

            # Draw bounding box
            cv2.rectangle(image, (left, top), (right, bottom), (0, 255, 0), 2)

            # Display label and confidence
            label = f"{COCO_CLASSES[class_indices[i]]}: {scores[i]:.2f}"
            cv2.putText(image, label, (left, top - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    # Show the result with matplotlib
    plt.figure(figsize=(12, 8))
    plt.imshow(image)
    plt.axis('off')
    plt.show()
```

## Step 6: Evaluating the model with the given image

The image path would be different for your image.

```python
# Example usage with your image path
image_path = "D:\\Deep Learning Course\\Codes\\Images for pretrained\\Sample image.jpg"
try:
    image_rgb, detected_boxes, detected_classes, detected_scores = detect_objects(image_path, model, threshold=0.5)
    display_image_with_detections(image_rgb, detected_boxes, detected_classes, detected_scores)
except FileNotFoundError as e:
    print(e)
```

# Tasks:

- Test it with your own image
- Take another two models with different backbones
- Take a bunch of data for evaluating and then find its mAP

Maths for Faster RCNN.

Stage 1: Forward Pass

The forward pass computes the object detection predictions in Faster R-CNN.

Step 1: Input Image → Feature Map via CNN

Input Image: Assume the input image is of size $256 \times 256$pixels.

$$\text{Input Image} = 256 \times 256$$

The image is passed through a pretrained CNN (e.g., VGG16 or ResNet) to extract a feature map of reduced dimensions. Assume the feature map $F$has dimensions $64 \times 64$.

$$F = \text{CNN(Input Image)} = 64 \times 64$$

Step 2: Region Proposal Network (RPN)

Anchor Box Generation

At each location in the feature map, we generate anchor boxes with different sizes and aspect ratios. Let's assume we generate 3 anchor boxes per location:

Anchor 1: $128 \times 128$(square)

Anchor 2: $256 \times 128$(horizontal)

Anchor 3: $128 \times 256$(vertical)

Given the feature map size of $64 \times 64$, the total number of anchor boxes is:

$$\text{Total Anchors} = 64 \times 64 \times 3 = 12{,}288 \text{ anchor boxes}$$

Objectness Score (Binary Classification)

For each anchor box, the RPN predicts the objectness score ($p_{\text{object}}$), which indicates the probability of the anchor containing an object. The objectness score is calculated using the sigmoid activation function.

$$p_{\text{object}} = \sigma(W_{\text{cls}} \cdot f_{\text{anchor}} + b_{\text{cls}})$$

Where:

$W_{\text{cls}}$is the classification weight learned during training.

$f_{\text{anchor}}$is the feature vector for the anchor at each location on the feature map.

$b_{\text{cls}}$is the bias term.

$\sigma$is the sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$.

Example objectness scores for the first few anchors might be:

Anchor 1: $p_{\text{object}} = 0.9$

Anchor 2: $p_{\text{object}} = 0.3$

Anchor 3: $p_{\text{object}} = 0.5$

Bounding Box Regression (Adjustment)

For each anchor, the RPN predicts bounding box adjustments $(\Delta x, \Delta y, \Delta w, \Delta h)$. These adjustments refine the position and size of the anchor box to make it match the ground truth box.

Given:

Ground truth box (GT): $(x_{\text{gt}}, y_{\text{gt}}, w_{\text{gt}}, h_{\text{gt}}) = (100,100,100,100)$

Anchor box (P): $(x_{\text{anchor}}, y_{\text{anchor}}, w_{\text{anchor}}, h_{\text{anchor}}) = (80,80,120,120)$

The formulas for calculating the adjustments are:

$$\Delta x = \frac{x_{\text{gt}} - x_{\text{anchor}}}{w_{\text{anchor}}}$$
$$\Delta y = \frac{y_{\text{gt}} - y_{\text{anchor}}}{h_{\text{anchor}}}$$
$$\Delta w = \log\left(\frac{w_{\text{gt}}}{w_{\text{anchor}}}\right)$$
$$\Delta h = \log\left(\frac{h_{\text{gt}}}{h_{\text{anchor}}}\right)$$

Let's compute these for Anchor 1:

$$\Delta x = \frac{100 - 80}{120} = 0.167$$
$$\Delta y = \frac{100 - 80}{120} = 0.167$$
$$\Delta w = \log\left(\frac{100}{120}\right) = -0.182$$
$$\Delta h = \log\left(\frac{100}{120}\right) = -0.182$$

Thus, the predicted bounding box adjustments for Anchor 1 are:

$$(\Delta x, \Delta y, \Delta w, \Delta h) = (0.167, 0.167, -0.182, -0.182)$$

Step 3: Region Proposal Generation

The RPN generates region proposals by applying these bounding box adjustments to the anchor boxes.

For Anchor 1, we apply the predicted adjustments:

$$x_{\text{pred}} = x_{\text{anchor}} + \Delta x \times w_{\text{anchor}} = 80 + 0.167 \times 120 = 100.04$$
$$y_{\text{pred}} = y_{\text{anchor}} + \Delta y \times h_{\text{anchor}} = 80 + 0.167 \times 120 = 100.04$$
$$w_{\text{pred}} = w_{\text{anchor}} \times \exp(\Delta w) = 120 \times \exp(-0.182) = 120 \times 0.833 = 99.96$$
$$h_{\text{pred}} = h_{\text{anchor}} \times \exp(\Delta h) = 120 \times \exp(-0.182) = 120 \times 0.833 = 99.96$$

The final refined bounding box for Anchor 1 is:

$$(x_{\text{final}}, y_{\text{final}}, w_{\text{final}}, h_{\text{final}}) = (100.04, 100.04, 99.96, 99.96)$$

This proposal is then passed to the ROI Pooling layer for further processing.

Step 4: ROI Pooling

After the region proposals are generated, they are passed through ROI Pooling. This process converts proposals of different sizes into fixed-size feature vectors (e.g., $7 \times 7$) that can be processed by fully connected layers in the detection network.

Step 5: Object Detection Network

The ROI pooled feature vectors are passed through the detection network, which:

Classifies the object inside each region proposal.

Refines the bounding box further.

Let's assume the model predicts:

Class 1 (Car) with probability $\hat{y}_1 = 0.9$

Class 2 (Person) with probability $\hat{y}_2 = 0.1$

And the bounding box adjustments are:

$$\Delta x = 0.05$$

$$\Delta y = 0.05$$

$$\Delta w = -0.1$$

$$\Delta h = -0.1$$

The final refined bounding box is:

$$(x_{\text{final}}, y_{\text{final}}, w_{\text{final}}, h_{\text{final}}) = (100.04 + 0.05, 100.04 + 0.05, 99.96 \times \exp(-0.1), 99.96 \times \exp(-0.1)) = (100.09, 100.09, 90.38, 90.38)$$

Backward Pass

After the forward pass, we compute the losses and backpropagate the gradients to update the weights.

Faster R-CNN uses two main losses in its training process:

RPN Loss: This includes:

Classification Loss: Binary cross-entropy loss for objectness classification (object vs. background).

Bounding Box Regression Loss: Smooth L1 loss for bounding box adjustments.

Detection Loss: This includes:

Classification Loss: Cross-entropy loss for object classification.

Bounding Box Regression Loss: Smooth L1 loss for refining bounding boxes.

We'll dive deeper into these losses, explaining what each variable means and how they contribute to the model's optimization.

1. RPN Loss (Region Proposal Network Loss)

The RPN generates region proposals using anchor boxes. The loss function for the RPN is the sum of two components:

Classification Loss: The RPN predicts whether each anchor box contains an object or not (object vs. background).

Bounding Box Regression Loss: The RPN predicts adjustments to the anchor boxes to make them more accurate.

RPN Classification Loss (Objectness Score)

The classification loss is a binary cross-entropy loss that calculates the error between the predicted objectness score and the ground truth label for each anchor box.

$$\mathcal{L}_{\text{cls}}^{\text{RPN}} = -\sum_{i} \quad y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)$$

Where:

$y_i$: Ground truth label for anchor $i$. This is either 1 (object) or 0 (background). The ground truth for each anchor box is determined by whether it overlaps sufficiently with a ground truth object (usually defined by an IoU threshold).

$\hat{y}_i$: Predicted objectness score for anchor $i$, i.e., the probability that the anchor contains an object. This value is predicted by the RPN using a sigmoid activation.

In simple terms:

$\hat{y}_i$represents the model's confidence that anchor $i$contains an object.

$y_i$is the true label, either 1 if the anchor box contains an object, or 0 if it's background.

RPN Bounding Box Regression Loss

The bounding box regression loss is used to refine the anchor boxes predicted by the RPN. This is a Smooth L1 Loss (also known as Huber loss) applied to the predicted adjustments $\Delta x, \Delta y, \Delta w, \Delta h$, which adjust the anchor box to better match the ground truth box.

$$\mathcal{L}_{\text{box}}^{\text{RPN}} = \sum_{i} \quad \text{smooth}_{L1}(\Delta x_i, \Delta y_i, \Delta w_i, \Delta h_i)$$

Where:

$\Delta x_i, \Delta y_i$: The predicted center adjustments for anchor $i$, calculated as:

$$\Delta x = \frac{x_{\text{gt}} - x_{\text{anchor}}}{w_{\text{anchor}}}, \Delta y = \frac{y_{\text{gt}} - y_{\text{anchor}}}{h_{\text{anchor}}}$$

These values are learned by the model to shift the center of the anchor box to better align with the ground truth center.

$\Delta w_i, \Delta h_i$: The predicted width and height adjustments for anchor $i$, calculated as:

$$\Delta w = \log\left(\frac{w_{\text{gt}}}{w_{\text{anchor}}}\right), \Delta h = \log\left(\frac{h_{\text{gt}}}{h_{\text{anchor}}}\right)$$

These values scale the anchor box's width and height to match the ground truth.

Smooth L1 Loss (Huber loss) is defined as:

$$\text{smooth}_{L1}(x) = \begin{cases} 0.5 \cdot x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{if } |x| \geq 1 \end{cases}$$

This loss function is less sensitive to outliers than the traditional L2 loss and helps the model learn more robust adjustments.

2. Detection Loss (for Object Classification and Bounding Box Regression)

After the RPN generates the region proposals, they are passed to the detection network, which performs object classification and bounding box refinement.

Detection Classification Loss (Object Class Prediction)

The detection network performs multi-class classification to predict the object class within each region proposal. This is calculated using softmax cross-entropy loss.

$$\mathcal{L}_{\text{cls}}^{\text{det}} = -\sum_{i} \quad y_i \log(\hat{y}_i)$$

Where:

$y_i$: Ground truth class label for region proposal $i$, representing the true object class. If the region contains a car, $y_i$could be 1, or if it contains a person, $y_i$could be 2, etc.

$\hat{y}_i$: Predicted probability for each class, predicted by the detection network for region proposal $i$using softmax activation.

Detection Bounding Box Regression Loss

Similar to the RPN, the detection network also predicts bounding box adjustments to refine the region proposals. The loss function used is also Smooth L1 Loss, which is applied to the predicted bounding box adjustments.

$$\mathcal{L}_{\text{box}}^{\text{det}} = \sum_{i} \quad \text{smooth}_{L1}(\Delta x_i, \Delta y_i, \Delta w_i, \Delta h_i)$$

Where:

$\Delta x_i, \Delta y_i$: The predicted center adjustments for the region proposal $i$.

$\Delta w_i, \Delta h_i$: The predicted width and height adjustments for the region proposal $i$.

These values are refined in the detection network after the region proposals are processed through ROI pooling.

Total Loss Function

The total loss in Faster R-CNN is the sum of:

RPN Loss (classification and bounding box regression)

Detection Loss (classification and bounding box regression)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{RPN}} + \mathcal{L}_{\text{det}}$$

Where:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{cls}}^{\text{RPN}} + \mathcal{L}_{\text{box}}^{\text{RPN}}$$
$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{cls}}^{\text{det}} + \mathcal{L}_{\text{box}}^{\text{det}}$$

Weight Updates

The weights of both the RPN and the detection network are updated using gradient descent. The weight update rule is:

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

Where:

$W_{\text{old}}$is the current weight.

$W_{\text{new}}$is the updated weight.

$\eta$is the learning rate.

$\frac{\partial \mathcal{L}}{\partial W}$is the gradient of the loss with respect to the weight.